

Algoritmi e Strutture Dati–parte I

lez. 7

Francesco Camastra

francesco.camastra@uniparthenope.it

Docente

- Prof. Francesco Camastra

Orario lezioni

- Mercoledì 11.00-13.00
(aula 11, Secondo Piano)
- Giovedì 9.00-11.00 (aula 11, Secondo Piano)

Ricevimento

- Orario: Mercoledì 14.00-18.00
- Dove: Stanza 430, Quarto piano, Centro Direzionale, Isola C4
- Per il ricevimento è **OBBLIGATORIO** inviare una email di prenotazione almeno due giorni prima del giorno di ricevimento.
- email del Docente:
francesco.camastra@uniparthenope.it
- Pagina del Docente:
dist.uniparthenope.it/francesco.camastra

Prerequisiti

- Programmazione I
- Matematica I
- Programmazione II
- **NOTA BENE:** Le propedeuticità non sono obbligatorie ... Ossia potreste anche sostenere l'esame senza aver dato Programmazione I e Programmazione II

Frequenza

- La Frequenza del corso non è obbligatoria

Modalità dell' esame

- Prova scritta di Algoritmi e Strutture Dati I ovvero Due prove intercorso (validità quattro appelli)
- Svolgimento di un progetto Algoritmi e Strutture Dati II da scegliersi tra una lista di progetti che verrà comunicata dal docente (validità quattro appelli).
- Superato lo scritto di ASD-I ed il progetto di ASD-II si accede all' Esame Orale congiunto di ASD-I e ASD-II

Testi Consigliati

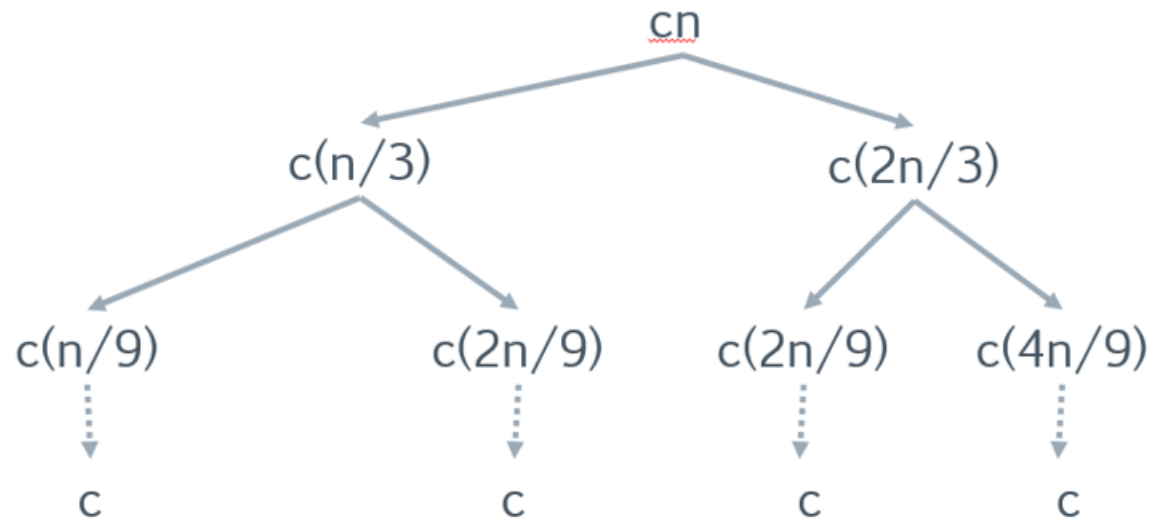
- ALGORITMI:
- Cormen, Leiserson, Rivest, Stein
*Introduzione agli Algoritmi e
alle strutture Dati*
(seconda edizione o terza edizione)
Ed. Mc Graw-Hill.

Metodo con Albero di Ricorsione: Un altro Esempio

- Consideriamo la ricorrenza.

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + O(n).$$

La scriviamo
come: $T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + cn$



(cont.)

- Quando sommiamo i valori nei singoli livelli dell'albero di ricorsione, otteniamo un valore pari a cn per ogni livello. L'altezza k dell'albero è data da $\left(\frac{2}{3}\right)^k n = 1$ ossia quando $k = \log_{\frac{3}{2}} n$. Perciò prevediamo che la soluzione della ricorrenza sia al massimo $O\left(cn \log_{\frac{3}{2}} n\right) = O(n \log n)$.
- Tuttavia non tutti i livelli dell'albero contribuiscono con un costo cn in quanto l'albero di ricorsione non è completo.

(cont.)

- Pertanto ha meno di $2^k = 2^{\log_3 n} = n^{\log_3 2}$ foglie.
Via via che si scende dalla radice, mancano sempre più nodi interni. Di conseguenza non tutti i livelli contribuiscono con un valore pari a cn . I livelli più bassi contribuiscono in misura minore. In realtà accettiamo l' approssimazione che sia un albero completo e proviamo che $T(n) = O(n \log n)$.

Verifica della ricorrenza

- Dimostriamo che $T(n) \leq dn \log n$. Abbiamo

$$\begin{aligned} T(n) &\leq T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + cn \leq d\frac{n}{3}\log\frac{n}{3} + d\frac{2n}{3}\log\frac{2n}{3} + cn \\ &\leq d\frac{n}{3}(\log n - \log 3) + d\frac{2n}{3}(\log 2n - \log 3) + cn \\ &\leq dn \log n - d\frac{n}{3}\log 3 + d\frac{2n}{3} - d\frac{2n}{3}\log n + cn \\ &\leq dn \log n - dn \log 3 + \frac{2}{3}dn + cn \\ &\leq dn \log n - \left(\log 3 - \frac{2}{3}\right)dn + cn \leq dn \log n \end{aligned}$$

(per d tale che $d \geq c / \left(\log 3 - \frac{2}{3}\right)$)

Paradigma DIVIDE-et-Impera

La tecnica di programmazione DIVIDE-et-Impera ha i seguenti passi:

- **DIVIDE** (il problema viene suddiviso in sottoproblemi)
- **IMPERA** (i problemi vengono risolti anche ricorsivamente)
- **COMBINA** (le soluzioni dei sottoproblemi vengono combinate)

DIVIDE-et-IMPERA è un approccio top-down

Quicksort

- Il Quicksort è l' algoritmo di ordinamento che generalmente prestazioni migliori tra quelli basati su confronto. È stato ideato da **Charles Antony Richard Hoare** nel 1960.
- È facilmente implementabile ed è un algoritmo general purpose, che ha un buon comportamento in un' ampia varietà di situazioni ed in molti casi richiede meno risorse di qualsiasi altro algoritmo.
- Gli svantaggi sono dati dal fatto che non è **stabile**, nel caso peggiore ha un comportamento quadratico.

(cont.)

- Sono stati svolti inoltre numerosi studi per migliorare le sue prestazioni. Praticamente dal momento in cui Hoare pubblicò per la prima volta il suo lavoro, la letteratura specializzata iniziò a proporre versioni migliorate dell'algoritmo.
- Sono state provate e analizzate molte idee, ma Quicksort è così ben bilanciato da far sì che il miglioramento apportato in una parte del programma quasi inevitabilmente dia luogo a un peggioramento delle prestazioni in qualche altro punto.
- Quicksort è stato scelto in molte librerie di linguaggi come il C di implementare di base una funzione che effettui l'ordinamento del Quicksort.

Quicksort

- È solitamente l' algoritmo di ordinamento più efficiente per insiemi con grande numero di elementi (cardinalità elevata)
- Input: un vettore di numeri $A_p, A_{p+1}, \dots, A_r$
- Output: Una permutazione del vettore di ingresso tale che $A_p^1 \leq \dots \leq A_r^1$

Algoritmo Quicksort

Adotta una strategia *divide-et-impera*

L' algoritmo consta di tre passi

- *Divide* nel quale il vettore di ingresso $A(p, r)$ viene suddiviso in due sottovettori $A(p, q)$ e $A(q+1, r)$ tali che ogni elemento del primo risulta essere minore uguale di ogni elemento del secondo (il valore q è prodotto funzione *PARTITION*)
- *Impera* nel quale i due sottovettori vengono ordinati con chiamate ricorsive all' algoritmo quicksort
- *Combina* nel quale i due sottovettori ordinati vengono fusi producendo l' output desiderato

Quicksort(A, p, r)

1. If $p < r$
2. $q \leftarrow \text{Partition}(A, p, r)$
3. Quicksort(A, p, q)
4. Quicksort($A, q + 1, r$)

Partition(A, p, r)

(versione originale Hoare)

$x \leftarrow A[p]$

$i \leftarrow p - 1$

$j \leftarrow r + 1$

While TRUE

do repeat $j \leftarrow j - 1$

until $A[j] \leq x$

do repeat $i \leftarrow i + 1$

until $A[i] \geq x$

if $i < j$ **then** scambia $A[i]$ con $A[j]$

else return j

Partition (un esempio)

5 3 6 4 4 1 6 2 7 (input)

5(i) 3 6 4 4 1 6 2(j) 7

2 3 6 4 4 1 6 5 7

2 3 6(i) 4 4 1(j) 6 5 7

2 3 1 4 4 6 6 5 7

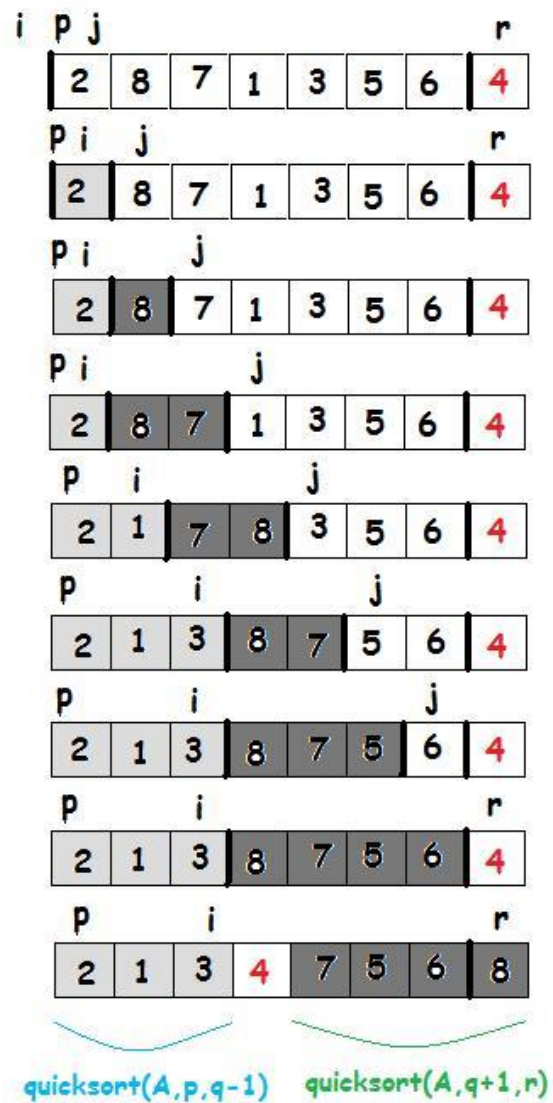
2 3 1 4 4(j) 6(i) 6 5 7

$A[p,q] = 2\ 3\ 1\ 4\ 4$

$A[q+1,r] = 6\ 5\ 7$

Partition(A, p, r)

1. $x \leftarrow A[r]$
2. $i \leftarrow p - 1$
3. **for** $j \leftarrow p$ to $r - 1$
4. **do if** $A[j] \leq x$
5. **then** $i \leftarrow i + 1$
6. scambia $A[i] \Leftrightarrow A[j]$
7. scambia $A[i + 1] \Leftrightarrow A[r]$
8. **return** $i + 1$



Partition

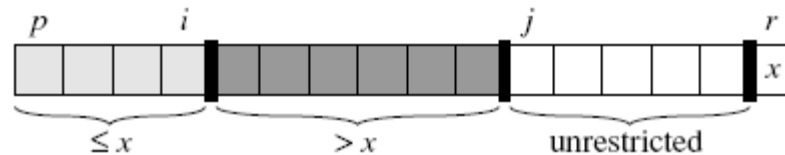
- Partition seleziona sempre un elemento $x = A[r]$ come pivot intorno al quale partizionare il sottoarray $A[p \dots r]$.

Partition: Invariante di Ciclo

- All' inizio di ogni iterazione del ciclo, per qualsiasi indice k dell' array, si ha:
 1. Se $p \leq k \leq i$, allora $A[k] \leq x$.
 2. Se $i + 1 \leq k \leq j - 1$, allora $A[k] > x$
 3. Se $k = r$, allora $A[k] = x$.

(cont.)

- Gli indici tra j e $r - 1$ non rientrano in alcuno dei tre casi e i corrispondenti valori in queste posizioni non hanno una particolare relazione con il pivot x .

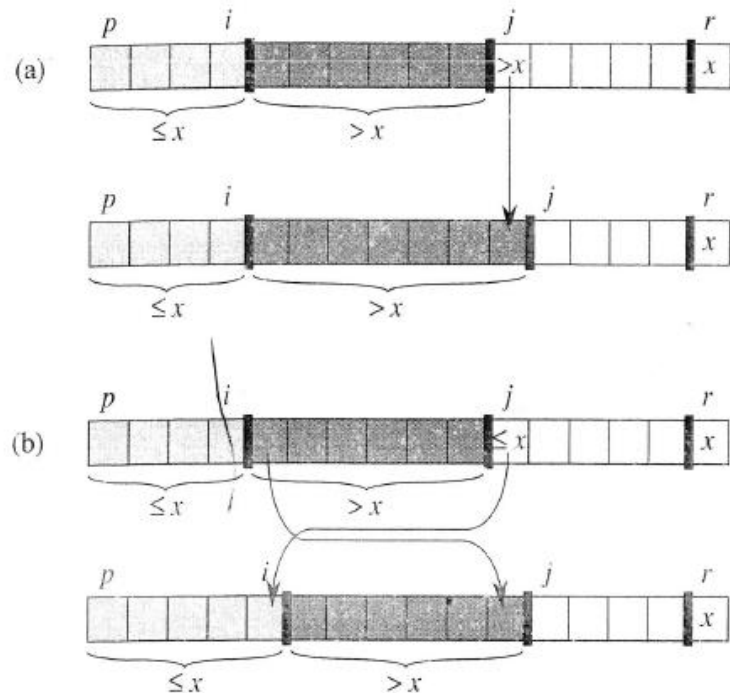


Invariante di Ciclo: Inizializzazione

- Prima della prima iterazione del ciclo, $i = p - 1$ e $j = p$. Non ci sono valori fra p e i né fra $i + 1$ e $j - 1$, quindi le prime due condizioni sono verificate.
- La terza condizione è assicurata dalla riga 1.

Invariante di Ciclo: Conservazione

- Ci sono due casi da considerare, a seconda del risultato del test della riga 4.



(cont.)

- Quando $A[j] > x$, si ricade nel caso a. L' unica azione , l' unica azione è incrementare j . Dopo l' incremento di j , la condizione 2 è soddisfatta per $A[j - 1]$ e tutte le altre posizioni non cambiano.
- Nell' altro caso, viene incrementato l' indice i , vengono scambiati $A[i]$ e $A[j]$ e, poi, viene incrementato l' indice j . Dopo lo scambio, abbiamo che $A[i] \leq x$ e la condizione 1 è soddisfatta. Inoltre abbiamo che $A[j - 1] > x$, in quanto l' elemento spostato è più grande di x .

Invariante di Ciclo:

Conclusione

- Alla fine del ciclo, $j = r$. Pertanto, ogni posizione dell' array si trova in uno dei tre insiemi descritti dall' invariante e noi abbiamo ripartito i valori dell' array in tre insiemi: quelli minori o uguali a x , quelli maggiori di x e un insieme di un solo elemento che contiene x .

Quicksort: Caso Peggior

- I dati sono ordinati in modo **decrescente** o **crescente**. In tal caso Partition produrrà sempre un secondo vettore costituito da un elemento.
- Esempio:
 - $A[p, r] = 1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9$
 - $A[p, q] = 1\ 2\ 3\ 4\ 5\ 6\ 7\ 8$
 - $A[q + 1, r] = 9$

In tal caso il tempo richiesto $T(n) = \Theta(n^2)$

Quicksort: Caso Peggior

$$T(n) = T(n - 1) + \Theta(n)$$

$$T(n) = T(n - 2) + \Theta(n) + \Theta(n - 1)$$

$$T(n) = T(n - 3) + \Theta(n) + \Theta(n - 1) + \Theta(n - 2)$$

.....

$$T(n) = \sum_{i=1}^n \Theta(i) = \Theta\left(\sum_{i=1}^n i\right) = \Theta\left(\frac{n(n+1)}{2}\right)$$

$$T(n) = \Theta(n^2)$$

Caso Peggior:

Metodo della Sostituzione

- Per semplicità riscriviamo la ricorrenza come:
 $T(n) = T(n - 1) + cn$.

- Verifichiamo prima che è $O(n^2)$. Dimostriamo che $T(n) \leq dn^2$. Sostituendo: $T(n) \leq d(n - 1)^2 + cn \leq d(n^2 - 2n + 1) + cn$

$$\leq dn^2 - 2dn + d + cn$$

$$\leq dn^2 - (2dn - d - cn) \leq dn^2$$

per ogni d tale che $2dn - d - c \geq 0$, ossia

$$\text{per } d \geq \frac{cn}{2n-1}.$$

(cont.)

- Verifichiamo che è $\Omega(n^2)$. Dimostriamo che è $T(n) \geq en^2$. Sostituendo: $T(n) \geq e(n-1)^2 + cn = e(n^2 - 2n + 1) + cn$
 $T(n) \geq en^2 - e(2n - 1) + cn \geq en^2$
per ogni e tale che $cn - e(2n - 1) \leq 0$ ossia per $e < \frac{cn}{2n-1}$.
- Poiché abbiamo verificato che è $T(n) = \Omega(n^2)$ ed è $T(n) = O(n^2)$, si ha **$T(n) = \Theta(n^2)$**

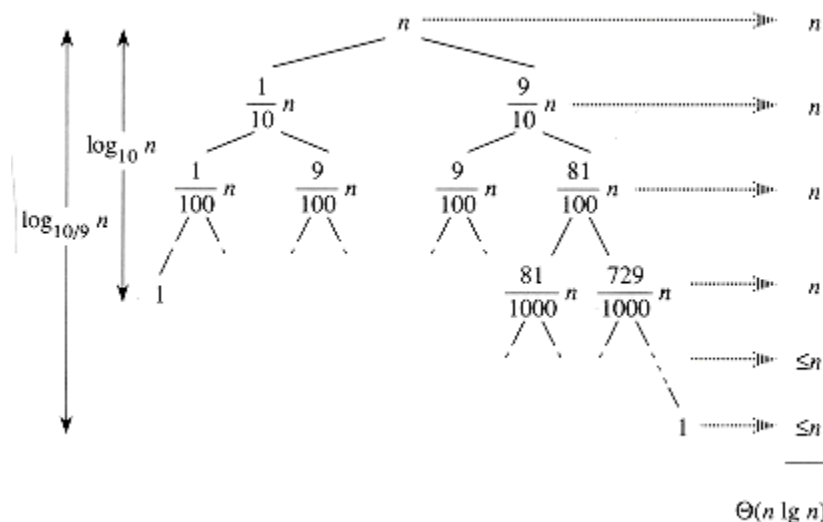
Quicksort: Caso Migliore

- Nel caso migliore, PARTITION produce due sottoproblemi, ciascuno di dimensione non maggiore di $\frac{n}{2}$, ossia uno ha dimensione $\left\lfloor \frac{n}{2} \right\rfloor$ e l'altro di dimensione $\left\lceil \frac{n}{2} \right\rceil$. La ricorrenza per il tempo di esecuzione diventa:
$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$
 che ha soluzione per il caso 2 del Teorema dell'esperto $T(n) = \Theta(n \log n)$.

Partizionamento Bilanciato

- Supponiamo che l' algoritmo di partizionamento produca sempre una ripartizione proporzionale 9 a 1. In questo caso abbiamo la ricorrenza:

$$T(n) \leq T\left(\frac{9n}{10}\right) + T\left(\frac{n}{10}\right) + cn$$



(cont.)

- Ogni livello ha un costo cn sino a quando non si raggiunge la profondità p tale che $\left(\frac{1}{10}\right)^p n = 1$, ossia quando $p = \log_{10} n$. A partire da quel livello i livelli hanno un costo $\leq cn$ ossia $O(n)$.
- La ricorsione termina alla profondità h tale che $\left(\frac{9}{10}\right)^h n = 1$, ossia quando $h = \log_{\frac{9}{10}} n = \Theta(\log n)$.
- Pertanto il costo totale del quicksort è **$O(n \log n)$** .

(cont.)

- Anche una ripartizione 99 a 1 determina come complessità $O(n \log n)$.
- La ragione è che qualsiasi ripartizione con proporzionalità costante produce un albero di ricorsione di profondità $\Theta(\log n)$, dove il costo di ogni livello è $O(n)$.
- Il tempo di esecuzione è quindi $O(n \log n)$ quando la ripartizione ha proporzionalità costante.

Considerazione

- Le prestazioni di Quicksort dipendono dal perno (poichè $x = A[r]$)
pertanto una buona pratica può essere quella di scambiare, prima di eseguire la **PARTITION**, il primo elemento con uno scelto a caso.

In questo caso si ha **RANDOMIZED_QUICKSORT**

RANDOMIZED_QUICKSORT(A, p, r)

1. **If** $p < r$
2. $q \leftarrow \text{Randomized_Partition}(A, p, r)$
3. $\text{Randomized_Quicksort}(A, p, q)$
4. $\text{Randomized_Quicksort}(A, q + 1, r)$

Randomized_Partition(A, p, r)

1. $i \leftarrow \text{Random}(p, r)$
2. Scambia $A[i]$ con $A[r]$
3. Partition(A, p, r)